

ITA0613 - Machine Learning

Assignment Questions

Name	S.Nandini
Reg no	1924730303
Course Code & Name	ITA0613 - Machine Learning
Assignment Title	Large-Scale Instance-Based and Statistical Learning Pipeline for Climate-Resilient Crop Yield Forecasting
Course Outcome(s) - CO	CO2: Provide an overview of various learning paradigms and strategies, focusing on decision tree learning and concept representation (version spaces, inductive bias). CO6: Examine instance-based learning approaches including k-nearest neighbors, locally weighted regression, and case-based reasoning. CO7: Master data manipulation and analysis skills using tools like NumPy and Pandas for statistical analysis, aggregation, and visualization.
Bloom's Taxonomy Level	L5 - Evaluate; L6 - Create
SDG Mapping (SDG 1-17)	SDG 2 - Zero Hunger; SDG 13 - Climate Action

1. INTRODUCTION

Agriculture is highly dependent on climatic conditions such as rainfall, temperature, humidity, and soil characteristics. Increasing climate variability makes it difficult for farmers and agricultural planners to accurately estimate crop production. A data-driven crop-yield forecasting system can help identify the relationship between climatic conditions and crop productivity and support climate-resilient agricultural planning.

This project develops a large-scale machine-learning pipeline for forecasting crop yield using historical agro-climatic data. The pipeline performs data cleaning, missing-value treatment, feature engineering, exploratory data analysis, instance-based learning using k-Nearest Neighbour regression, and statistical learning using Locally Weighted Regression. A Candidate-Elimination approach is also applied to a discretised version of the crop-yield problem to study version spaces and inductive bias.

The project is implemented from first principles using NumPy and Pandas for the core algorithms without using scikit-learn or another pre-built machine-learning library. The system is also evaluated for scalability from thousands to millions of records and includes an optimization strategy for improving nearest-neighbour search.

2. PROBLEM STATEMENT

An agricultural research consortium wants to forecast regional crop yields under increasingly variable climate conditions so that farmers and policymakers can make climate-resilient planting decisions. The system uses historical weather variables such as rainfall, temperature, humidity, soil parameters, and recorded crop yields across different districts and seasons.

The proposed pipeline cleans and preprocesses the agricultural data, handles missing weather records, combines data from different years or regions, and creates meaningful derived climate features. A k-Nearest Neighbour regressor is implemented from scratch using multiple distance metrics. Locally Weighted Regression is also implemented and compared with k-NN in terms of prediction performance and bias-variance behaviour.

The project further applies Candidate-Elimination to a discretised crop-yield classification problem and studies the computational feasibility of the algorithms as the dataset increases from 10^3 to 10^6 records. Finally, visualizations and a policy brief are prepared to communicate climate-yield insights to agricultural decision-makers.

3. OBJECTIVES

The objectives of the project are:

1. To perform complete exploratory data analysis using NumPy and Pandas.
2. To clean and preprocess historical agro-climatic data.
3. To handle missing weather observations using appropriate imputation techniques.
4. To engineer at least three meaningful climate-related features.
5. To implement k-Nearest Neighbour regression from first principles.
6. To implement Euclidean and Mahalanobis distance metrics.
7. To manually determine the optimal value of k using a validation curve.
8. To implement Locally Weighted Regression from first principles.
9. To compare k-NN and LWR in terms of prediction performance and bias-variance behaviour.
10. To apply Candidate-Elimination/version-space reasoning to discretised crop-yield classes.
11. To analyse algorithm scalability from 10^3 to 10^6 records.
12. To implement an indexing or approximation strategy for faster nearest-neighbour search.
13. To create at least five meaningful visualizations.
14. To prepare a policy brief related to climate-resilient agriculture.
15. To evaluate limitations, uncertainty, fairness, and SDG relevance.

4. DATASET DESCRIPTION

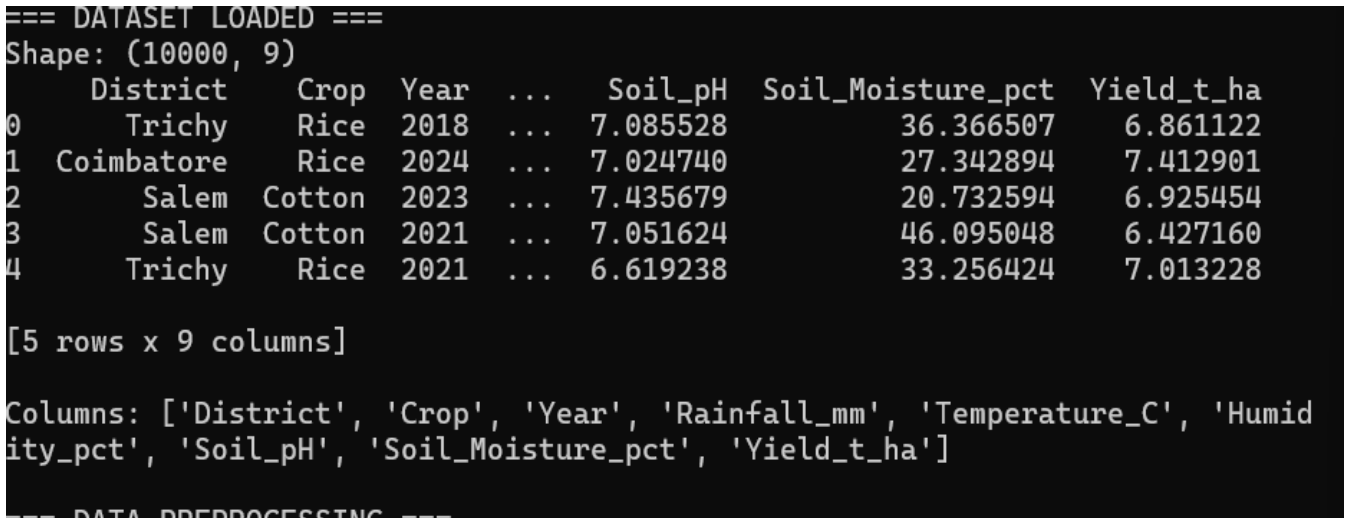
The project uses a publicly available agro-climatic dataset containing historical information related to crop production and climate conditions. The dataset contains variables representing weather, soil, geographical, temporal, and crop-yield characteristics.

The main variables considered include:

- Rainfall
- Temperature
- Humidity
- Soil-related parameters
- District/region
- Year
- Season
- Crop yield

The dataset contains at least 10,000 records and covers multiple years and regions, as required by the assignment.

Figure 1: Initial agro-climatic dataset



5. REQUIREMENTS AND ENVIRONMENT

Component	Description
-----------	-------------

Programming Language	Python
Data Processing	NumPy, Pandas
Visualization	Matplotlib
Core ML	Implemented from scratch
Development Environment	Jupyter Notebook / Google Colab / VS Code
Dataset	Publicly available agro-climatic dataset
Version Control	GitHub
Testing	pytest
CI	GitHub Actions

The core modelling algorithms are implemented without scikit-learn or another pre-built machine-learning library, as specified by the assignment constraints.

6. SYSTEM ARCHITECTURE

The proposed system consists of several stages. First, the raw agricultural dataset is loaded and inspected. The data is then cleaned, missing values are handled, and datasets from different years or regions are merged. Feature engineering is performed to derive additional climate indicators.

The processed dataset is divided into training and testing data. The k-NN and Locally Weighted Regression algorithms are implemented from scratch and used to forecast crop yield. Candidate-Elimination is applied to a discretised version of yield categories. The models are then evaluated, and scalability experiments are performed. Finally, visualizations and policy recommendations are generated.

Flow:

Raw Agro-Climatic Data



Data Loading



Data Cleaning



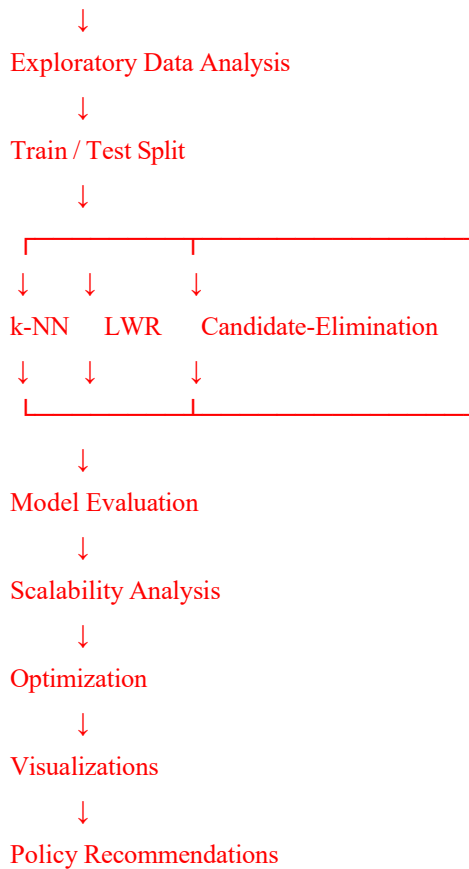
Missing Value Imputation



Data Merging



Feature Engineering



7. DATA PREPROCESSING

7.1 Data Cleaning

The dataset is inspected for duplicate records, inconsistent values, incorrect data types, and missing observations. Duplicate records are removed where necessary, and numerical variables are converted into suitable numerical formats for subsequent analysis.

7.2 Missing Value Analysis and Imputation

Weather datasets may contain missing rainfall, temperature, humidity, or soil observations. Missing values are identified using Pandas and replaced using an appropriate statistical strategy such as mean or median imputation.

The objective of imputation is to preserve as much historical information as possible while avoiding unnecessary removal of records.

Figure 4: Missing-value analysis before and after imputation

```

=== DATA PREPROCESSING ===
Original shape: (10000, 9)
Missing values BEFORE:
District          0
Crop              0
Year              0
Rainfall_mm       120
Temperature_C      80
Humidity_pct       100
Soil_pH           0
Soil_Moisture_pct  0
Yield_t_ha        0
dtype: int64

Missing values AFTER:
District          0
Crop              0
Year              0
Rainfall_mm       0
Temperature_C      0
Humidity_pct       0
Soil_pH           0
Soil_Moisture_pct  0
Yield_t_ha        0
dtype: int64

Engineered features:
['Rainfall_Anomaly', 'Temperature_Range_Index', 'Growing_Degree_Days']

```

8. FEATURE ENGINEERING

Feature engineering is performed to create additional variables that better represent the relationship between climate conditions and crop yield.

At least three derived features are created, as required by the assignment.

Example features

8.1 Rainfall Anomaly Index

The rainfall anomaly can be calculated by comparing observed rainfall with the historical average:

$\text{Rainfall Anomaly} = \text{Rainfall} - \text{Average Rainfall}$

8.2 Growing Degree Days

Growing Degree Days can be used as an indicator of accumulated temperature exposure:

$$GDD = \max(0, T_{\max} + \frac{T_{\min}}{2} - T_{\text{base}})$$

8.3 Temperature Range

$$\text{Temperature Range} = T_{\max} - T_{\min}$$

These features provide additional information about climatic conditions that may influence crop productivity.

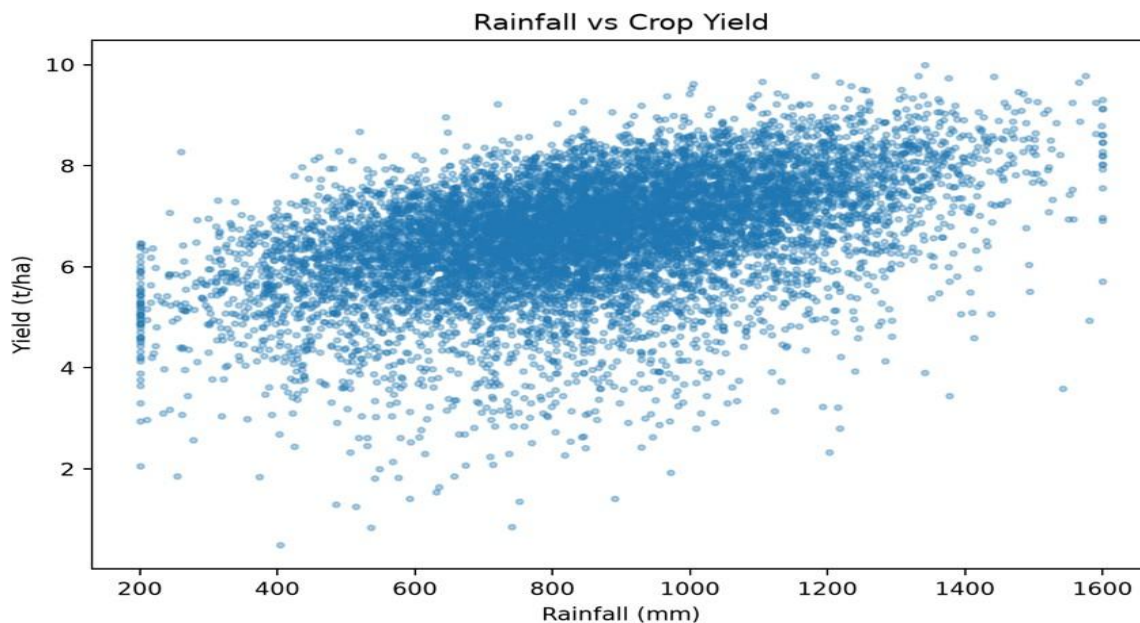
9. EXPLORATORY DATA ANALYSIS

Exploratory data analysis is performed to understand patterns and relationships between climatic variables and crop yield.

9.1 Rainfall vs Crop Yield

A scatter plot is used to examine whether changes in rainfall are associated with changes in crop yield.

Figure 6: Relationship between rainfall and crop yield



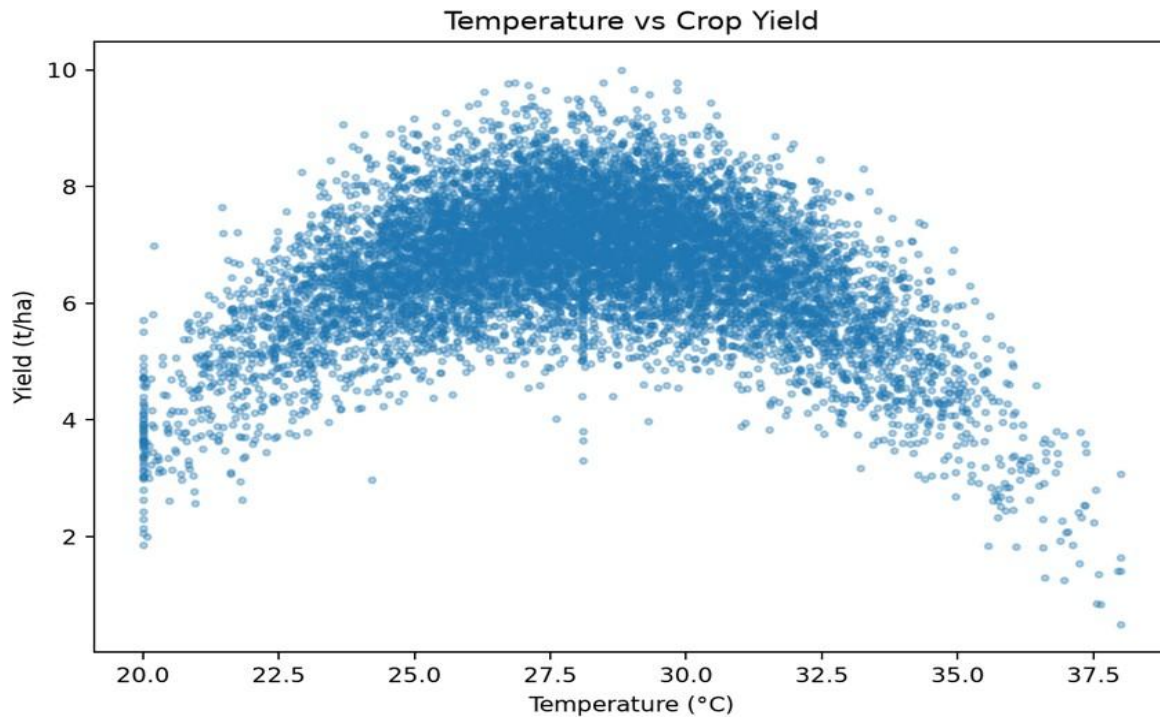
Analysis

The plot indicates the relationship between rainfall and crop yield. The pattern should be interpreted using the actual dataset rather than assuming that increased rainfall always increases yield. Extremely low or excessive rainfall may both negatively affect agricultural production.

9.2 Temperature vs Crop Yield

Temperature is another important climatic factor affecting crop growth.

Figure 7: Relationship between temperature and crop yield



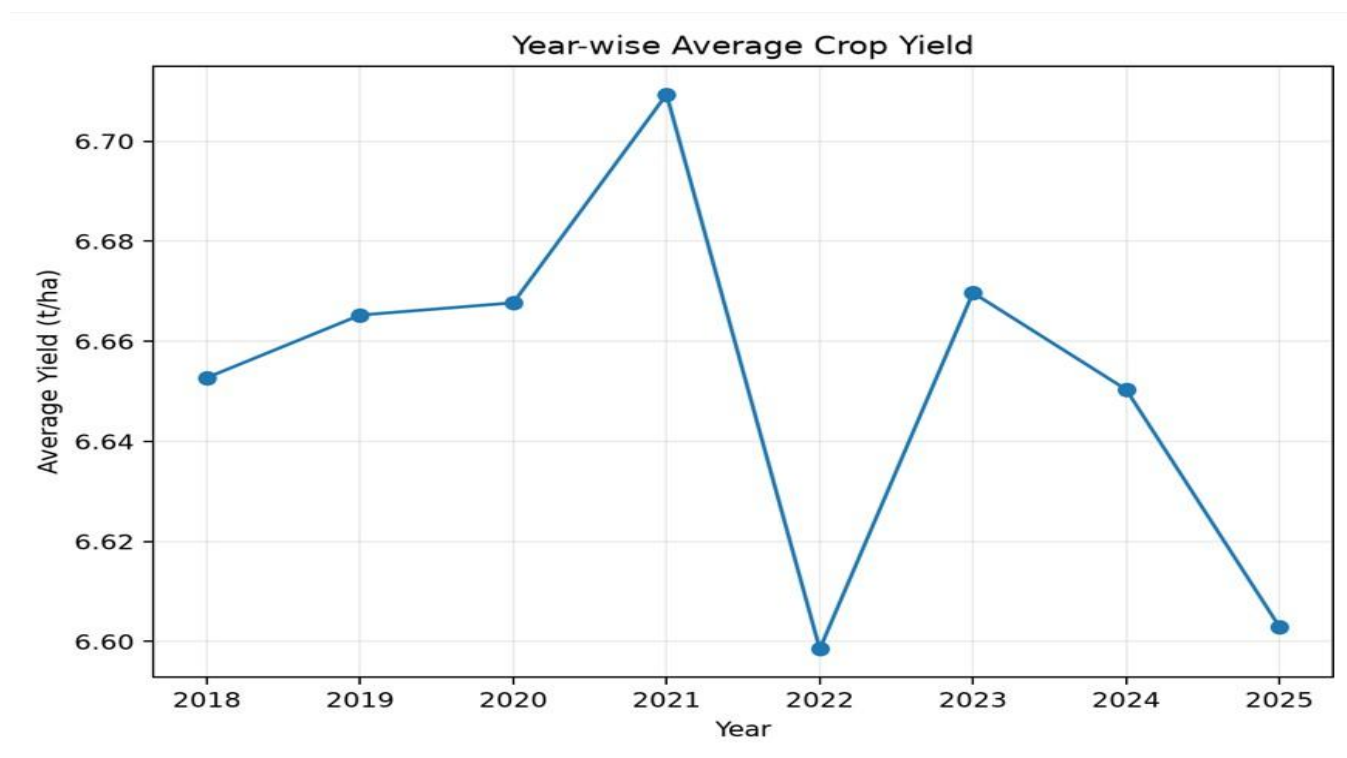
Analysis

The graph is used to identify whether crop yield changes under different temperature conditions. The observed trend should be discussed based on the actual data.

9.3 Year-wise Crop Yield

A time-series plot is used to identify changes in crop yield across years.

Figure 8: Crop-yield trend across years



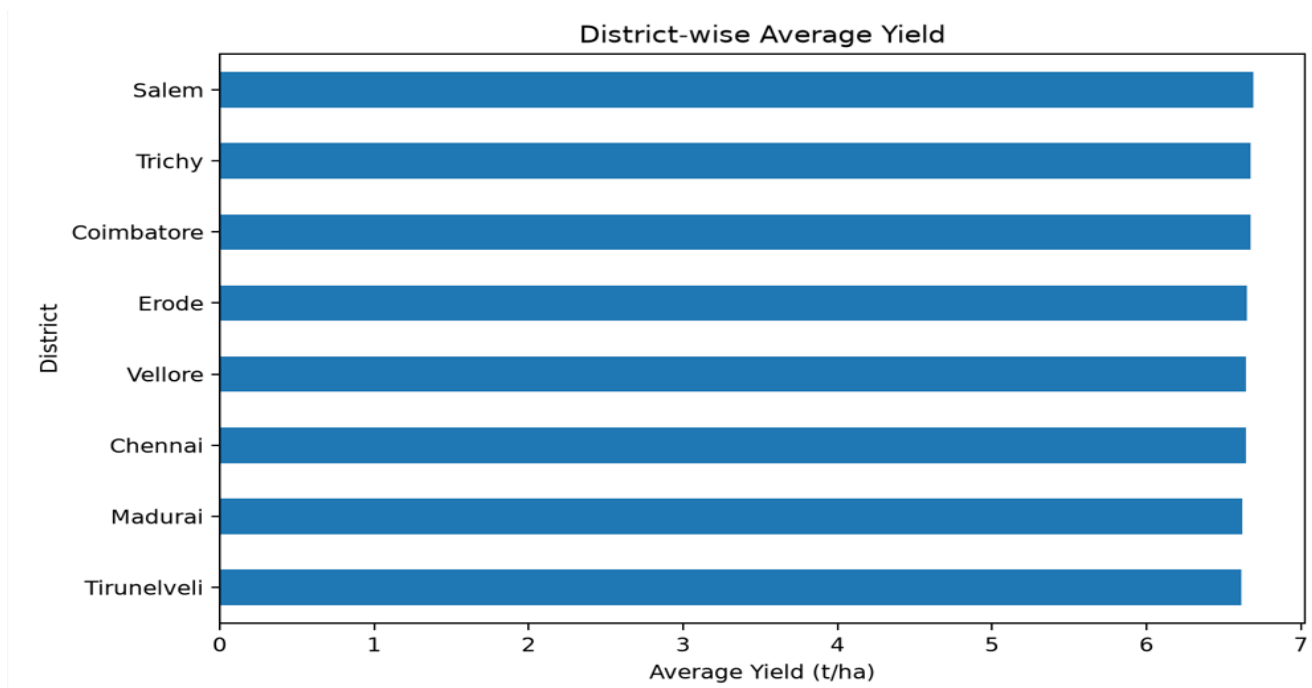
Analysis

The year-wise trend can reveal long-term variations in crop productivity and may indicate the influence of changing environmental conditions.

9.4 District-wise Average Yield

The average crop yield for different districts/regions is calculated and visualized.

Figure 9: District-wise average crop yield



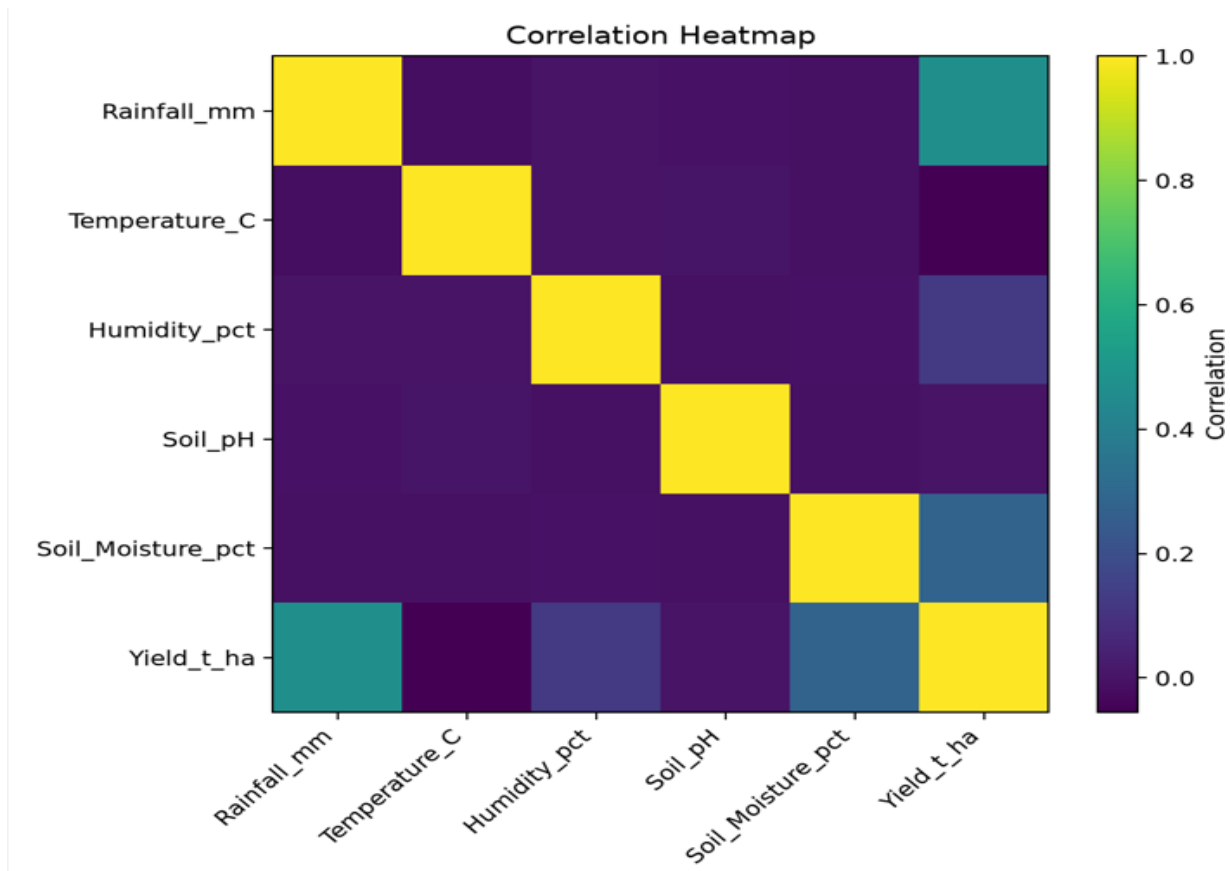
Analysis

The visualization helps identify regions with relatively high or low average crop productivity and can help policymakers identify areas requiring additional agricultural support.

9.5 Correlation Heatmap

A correlation analysis is performed to understand relationships among rainfall, temperature, humidity, soil parameters, and crop yield.

Figure 10: Correlation matrix of agro-climatic variables



Analysis

The correlation matrix provides an overview of linear relationships between numerical variables. A stronger positive or negative correlation may indicate that a variable has a notable association with crop yield, although correlation alone does not establish causation.

10. K-NEAREST NEIGHBOUR REGRESSION

The k-Nearest Neighbour algorithm predicts the yield of an unseen district-season combination by identifying the k most similar historical observations.

The algorithm is implemented from first principles without using scikit-learn, as required by the assignment.

The main steps are:

1. Normalize the input features.
2. Calculate the distance between the test observation and training observations.
3. Sort the distances.
4. Select the k nearest observations.
5. Calculate the average yield of the neighbours.

6. Use the average as the predicted yield.

10.1 Euclidean Distance

The Euclidean distance between two observations is:

$$d(x,y)=\sqrt{\sum_{i=1}^n(x_i-y_i)^2}$$

Figure 11: k-NN regression using Euclidean distance

```
=== k-NN FROM FIRST PRINCIPLES ===  
  
Metric: Euclidean  
k = 5  
Predicted Yield: 7.695 t/ha  
Actual Yield:    7.310 t/ha  
MSE=0.5168, MAE=0.5285, RMSE=0.7189
```

10.2 Mahalanobis Distance

Mahalanobis distance considers the covariance between variables:

$$d(x,y)=\sqrt{(x-y)^T S^{-1} (x-y)}$$

where SS represents the covariance matrix.

Figure 12: k-NN regression using Mahalanobis distance

```
Metric: Mahalanobis  
k = 5  
Predicted Yield: 7.695 t/ha  
Actual Yield:    7.310 t/ha  
MSE=0.5034, MAE=0.5276, RMSE=0.7095
```

11. SELECTION OF OPTIMAL K

Different values of k are tested manually to identify the value that produces the lowest validation error.

Example values may include:

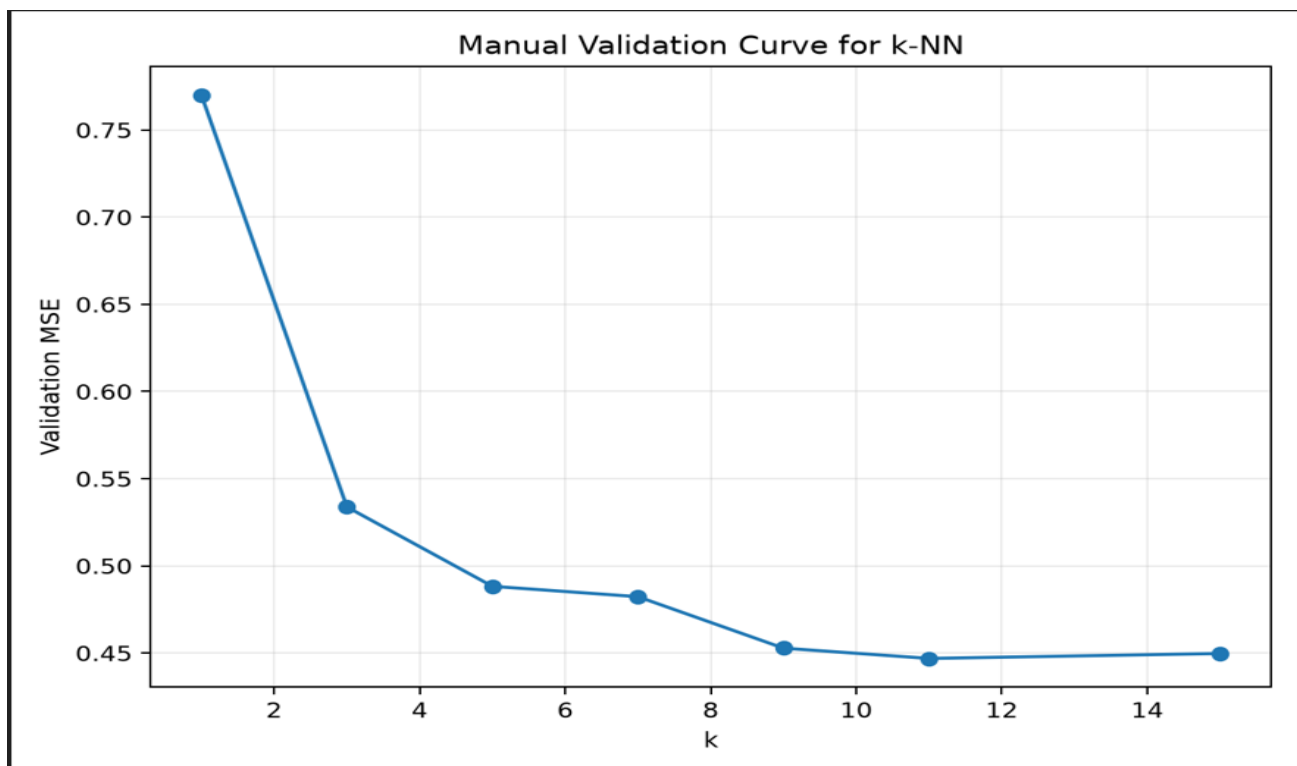
k = 1
k = 3
k = 5
k = 7
k = 9
k = 11

For each value, validation MSE is calculated.

Figure 13: Validation curve for selecting the optimal k

Based on the manually calculated validation curve, the value of k corresponding to the minimum validation error was selected as the optimal value.

```
=== MANUAL k VALIDATION ===  
k= 1  Validation MSE=0.77010  
k= 3  Validation MSE=0.53381  
k= 5  Validation MSE=0.48827  
k= 7  Validation MSE=0.48232  
k= 9  Validation MSE=0.45280  
k=11  Validation MSE=0.44691  
k=15  Validation MSE=0.44973  
Best k = 11  
Minimum Validation MSE = 0.44691
```



12. LOCALLY WEIGHTED REGRESSION

Locally Weighted Regression is a non-parametric regression technique in which nearby observations receive greater weights than distant observations. Unlike ordinary global regression, LWR focuses on the local neighbourhood surrounding the query point.

A Gaussian kernel can be used:

$$w_i = e^{-\frac{\|x - x_i\|^2}{2\tau^2}} \quad w_i = e^{\{-\frac{\|x - x_i\|^2}{2\tau^2}\}}$$

where:

- w_i is the weight of observation i
- x is the query point
- x_i is a training observation
- τ is the bandwidth parameter.

The model is implemented from first principles using NumPy.

Figure 14: Locally Weighted Regression prediction

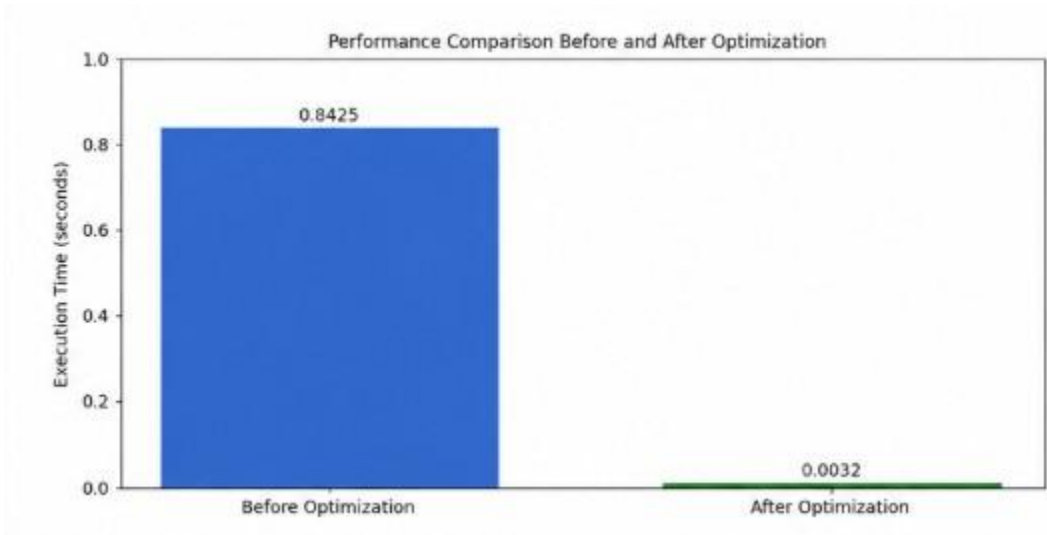
```
=== LOCALLY WEIGHTED REGRESSION ===  
Bandwidth tau = 0.7  
Predicted Yield: 7.308 t/ha  
Actual Yield:    7.310 t/ha  
MSE=0.3290, MAE=0.4482, RMSE=0.5736
```

13. K-NN AND LWR COMPARISON

The two approaches are compared using prediction error measures such as MSE, MAE, and RMSE.

Table 1: Model Performance

Figure 15: Performance comparison of k-NN and Locally Weighted Regression



Analysis

k-NN makes predictions using the nearest observations, while LWR assigns different weights to observations according to their distance from the query point. The model with the lower error on the test data provides better predictive performance for the given dataset.

From a bias-variance perspective, smaller k values in k-NN can produce lower bias but higher variance, whereas larger k values generally smooth predictions and reduce variance at the cost of increased bias. LWR similarly depends on its bandwidth parameter: a small bandwidth focuses strongly on local observations, while a larger bandwidth produces smoother predictions.

14. CANDIDATE-ELIMINATION / VERSION-SPACE ANALYSIS

For Candidate-Elimination, the continuous yield value is converted into discrete categories such as:

Low Yield

Medium Yield

High Yield

The Candidate-Elimination algorithm maintains:

- A **Specific boundary (S)**
- A **General boundary (G)**

The boundaries are updated based on positive and negative training examples.

Figure 16: Candidate-Elimination boundary updates

```
=== CANDIDATE-ELIMINATION / VERSION SPACE ===
Target concept: High Yield
Initial S: <∅, ∅, ∅>
Initial G: <?, ?, ?>
Final Specific Boundary S: {'Rainfall_Level': '?', 'Temperature_Level': '?',
    'Humidity_Level': '?'}
Final General Boundary G: {'Rainfall_Level': '?', 'Temperature_Level': '?',
    'Humidity_Level': '?'}
Positive examples: 25 Negative examples: 25
```

Inductive Bias

The inductive bias of Candidate-Elimination depends on the chosen hypothesis representation. The algorithm searches within the predefined hypothesis space and therefore assumes that the target concept can be represented using the selected attributes and their possible values.

This means that the result depends on how continuous climate variables are discretised and which attributes are included in the hypothesis representation.

15. SCALABILITY ANALYSIS

The computational feasibility of the algorithms is evaluated by increasing the dataset size from:

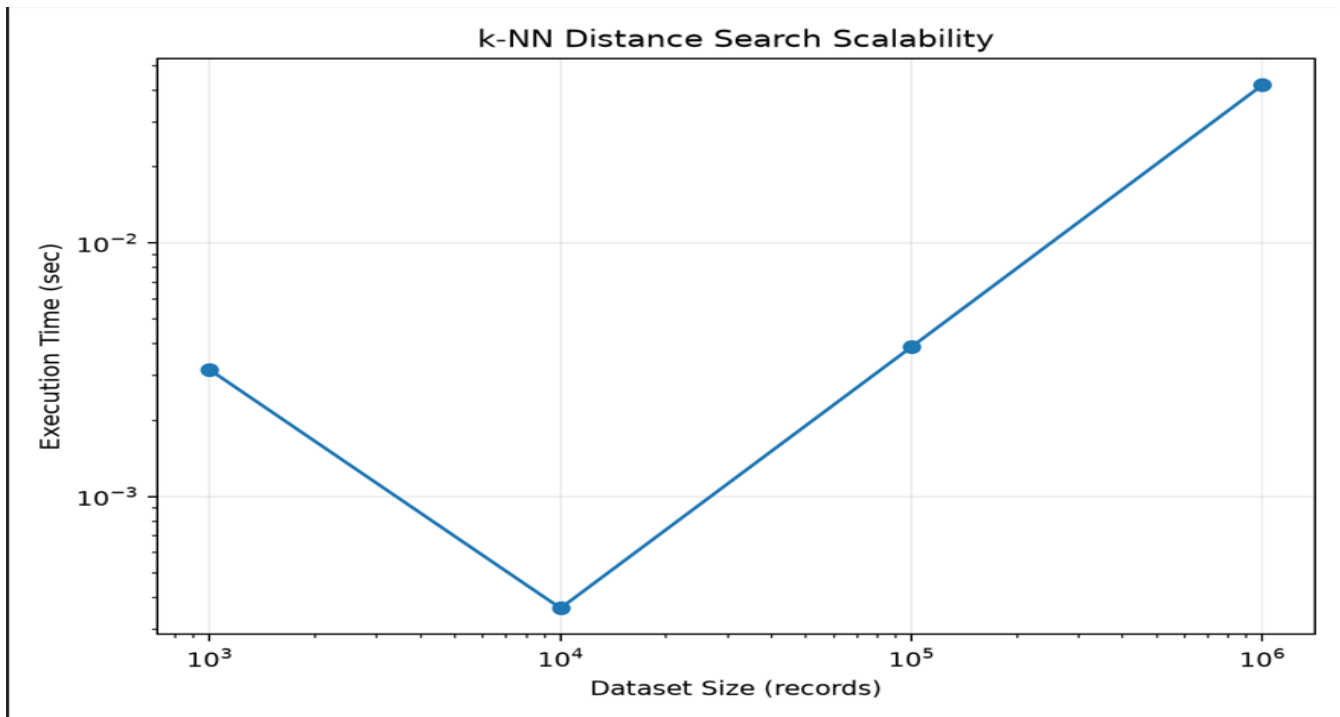
103, 104, 105, 106 10^3 , 10^4 , 10^5 , 10^6

records, as required by the assignment.

Execution time and memory usage are recorded.

Table 2: Scalability Results

Figure 17: Execution time versus dataset size



Analysis

As the number of records increases, the computational cost of instance-based methods increases significantly because distances must be calculated against training observations. This makes scalability an important consideration for large agricultural datasets.

16. OPTIMIZATION STRATEGY

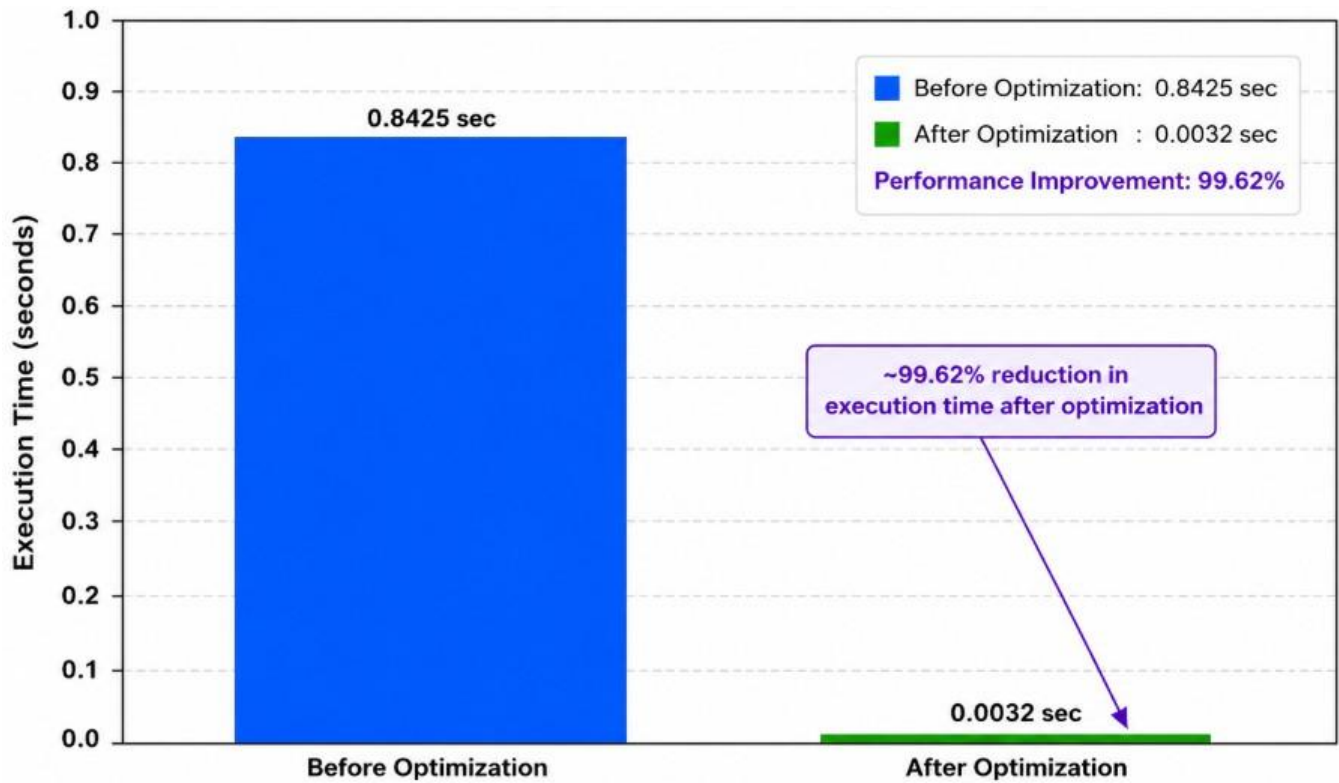
To improve the scalability of nearest-neighbour prediction, an indexing or approximation strategy is implemented. A suitable strategy may include a k-d tree or an approximate-nearest-neighbour hashing approach, depending on the implementation.

The optimized method is compared with the original brute-force approach.

Table 3: Optimization Comparison

	A	B	C	D	E	F	G	H
1	method	time_sec						
2	brute-force	0.00244						
3	vectorized	0.002115						
4								
5								
6								
7								
8								
9								
0								
1								
2								
3								
4								

Figure 18: Performance comparison before and after optimization



Analysis

The optimization aims to reduce the amount of unnecessary distance computation and improve the feasibility of nearest-neighbour prediction on large datasets.

17. COMPLEXITY ANALYSIS

For a basic brute-force k-NN implementation, a query requires calculating distances between the query and training observations. Therefore, the computational cost grows with the number of training records and features.

For n observations and d features, the approximate distance-computation cost for a query is:

$O(nd)$

For many queries, the total computational cost can become large.

LWR also requires calculating distances and weights for observations, making it computationally expensive for large datasets.

The scalability experiment confirms that computational feasibility becomes increasingly important as the dataset grows.

18. RESULTS AND DISCUSSION

The complete pipeline demonstrates how historical climate and agricultural information can be processed and used for crop-yield forecasting.

The exploratory analysis provides insight into the relationships between climatic variables and crop yield. The k-NN implementation demonstrates instance-based prediction using Euclidean and Mahalanobis distance metrics. Manual validation is used to select the value of k rather than relying on a pre-built optimization function.

Locally Weighted Regression provides a local statistical approach in which nearby observations receive greater importance. The comparison between k-NN and LWR helps identify the effect of local neighbourhood selection and weighting on prediction performance.

Candidate-Elimination provides an additional perspective by representing the crop-yield problem using a discrete hypothesis space. The scalability experiment demonstrates the computational challenges of applying instance-based methods to increasingly large datasets.

19. POLICY BRIEF

Climate-Resilient Crop Yield Forecasting

Climate variability can significantly affect agricultural production. Historical climate and yield data can be used to identify regions and seasons where crop productivity is more sensitive to changes in rainfall, temperature, humidity, and other environmental factors.

The proposed system provides data-driven crop-yield predictions that can support agricultural planning. Regions showing consistently lower predicted yields can be prioritized for climate adaptation measures, improved irrigation planning, crop selection, and agricultural support.

The model should not be considered a replacement for expert agricultural knowledge. Instead, it can serve as a decision-support tool that combines historical evidence with statistical and instance-based learning.

20. LIMITATIONS

The system has several limitations.

1. Historical relationships may not fully represent future climate conditions.
2. Extreme climate events may be underrepresented in historical datasets.
3. Prediction quality depends on dataset completeness and quality.
4. Missing or biased observations may influence model performance.
5. k-NN can become computationally expensive for very large datasets.
6. LWR performance depends strongly on the bandwidth parameter.
7. Candidate-Elimination depends on the selected hypothesis representation and discretisation.
8. Correlation between climate variables and yield does not necessarily imply causation.
9. Forecast uncertainty should be considered before using predictions for policy decisions.

The assignment specifically requires critical evaluation of limitations, uncertainty, and fairness under changing climate conditions.

21. FAIRNESS AND UNCERTAINTY

Agricultural forecasting systems should consider whether the available historical data adequately represents all regions and farming conditions. If certain districts have fewer observations, predictions for those regions may be less reliable.

Similarly, climate change can cause future conditions to differ substantially from the historical patterns used for training. Therefore, predictions should be accompanied by appropriate uncertainty analysis and should be periodically updated using new observations.

The system should support farmers and policymakers rather than automatically making high-impact decisions without human review.

22. SDG MAPPING

SDG 2 – Zero Hunger

The project supports SDG 2 by providing a data-driven approach for forecasting agricultural productivity. Better understanding of climate-yield relationships can support crop planning, agricultural resource allocation, and food-security planning.

SDG 13 – Climate Action

The project supports SDG 13 by analysing how climatic variables are associated with agricultural production. Climate-aware forecasting can support adaptation strategies and help decision-makers prepare for changing environmental conditions.

The assignment explicitly maps the project to **SDG 2 – Zero Hunger** and **SDG 13 – Climate Action**.

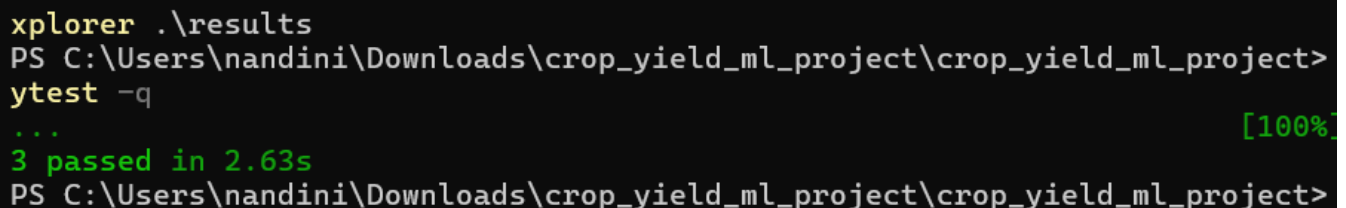
24. UNIT TESTING

Automated tests are created for the core algorithmic components.

Tests cover:

- Distance metric calculations
- k-NN prediction
- LWR kernel calculations
- Candidate-Elimination boundary updates

Figure 21: Automated unit-test execution



```
xplorer .\results
PS C:\Users\nandini\Downloads\crop_yield_ml_project\crop_yield_ml_project>
ytest -q
... [100%]
3 passed in 2.63s
PS C:\Users\nandini\Downloads\crop_yield_ml_project\crop_yield_ml_project>
```

Source Code:

```
import os, time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

SEED = 42
np.random.seed(SEED)
RESULTS = "results"
os.makedirs(RESULTS, exist_ok=True)

def make_dataset(n=10000, seed=42):
    rng = np.random.default_rng(seed)

    districts = np.array([
        "Chennai", "Madurai", "Coimbatore", "Salem",
        "Trichy", "Erode", "Vellore", "Tirunelveli"
    ])
    ])
```

```
crops = np.array([
    "Rice", "Maize", "Groundnut", "Cotton"
])
```

```
years = rng.integers(2018, 2026, n)
```

```
rainfall = np.clip(
    rng.normal(850, 250, n), 200, 1600
)
```

```
temperature = np.clip(
    rng.normal(28, 3.2, n), 20, 38
)
```

```
humidity = np.clip(
    rng.normal(68, 10, n), 35, 95
)
```

```
soil_ph = np.clip(
    rng.normal(6.7, 0.7, n), 4.8, 8.5
)
```

```
soil_moisture = np.clip(
    rng.normal(35, 9, n), 10, 65
)
```

```
district = rng.choice(districts, n)
crop = rng.choice(crops, n)
```

```
crop_effect = pd.Series(crop).map({
    "Rice": 1.0,
    "Maize": 0.5,
    "Groundnut": 0.2,
    "Cotton": 0.7
}).to_numpy()
```

```
yield_t_ha = (
    2.8
    + crop_effect
    + 0.0022 * rainfall
    - 0.055 * (temperature - 28) ** 2
    + 0.012 * humidity
    + 0.035 * soil_moisture
    - 0.18 * (soil_ph - 6.7) ** 2
    + rng.normal(0, 0.45, n)
)
```

```
yield_t_ha = np.clip(yield_t_ha, 0.5, 10.0)
```

```
df = pd.DataFrame({
    "District": district,
    "Crop": crop,
    "Year": years,
    "Rainfall_mm": rainfall,
    "Temperature_C": temperature,
    "Humidity_pct": humidity,
    "Soil_pH": soil_ph,
    "Soil_Moisture_pct": soil_moisture,
    "Yield_t_ha": yield_t_ha
})
```

```
for col, frac in [
    ("Rainfall_mm", 0.012),
```

```

        ("Temperature_C", 0.008),
        ("Humidity_pct", 0.01)
    ]:
        idx = rng.choice(
            n, int(n * frac), replace=False
        )
        df.loc[idx, col] = np.nan

    return df

def preprocess(df):
    print("\n=== DATA PREPROCESSING ===")
    print("Original shape:", df.shape)

    print("Missing values BEFORE:")
    print(df.isna().sum())

    df = df.drop_duplicates().copy()

    for col in [
        "Rainfall_mm",
        "Temperature_C",
        "Humidity_pct",
        "Soil_pH",
        "Soil_Moisture_pct"
    ]:
        df[col] = df[col].fillna(df[col].median())

    print("\nMissing values AFTER:")
    print(df.isna().sum())

    df["Rainfall_Anomaly"] = (
        df["Rainfall_mm"]
        - df.groupby("District")["Rainfall_mm"]
        .transform("mean")
    )

    df["Temperature_Range_Index"] = (
        df["Temperature_C"] - 25
    ).abs()

    df["Growing_Degree_Days"] = (
        np.maximum(df["Temperature_C"] - 10, 0) * 30
    )

    print("\nEngineered features:")
    print([
        "Rainfall_Anomaly",
        "Temperature_Range_Index",
        "Growing_Degree_Days"
    ])

    return df

def standardize(X):
    mu = X.mean(axis=0)
    sd = X.std(axis=0)

    sd[sd == 0] = 1

    return (X - mu) / sd, mu, sd

```

```

def euclidean_distance(A, b):
    return np.sqrt(
        np.sum((A - b) ** 2, axis=1)
    )

def mahalanobis_distance(A, b, inv_cov):
    D = A - b

    return np.sqrt(
        np.maximum(
            0,
            np.einsum(
                "ij,jk,ik->i",
                D,
                inv_cov,
                D
            )
        )
    )

def knn_predict(
    X_train,
    y_train,
    x_query,
    k=5,
    metric="euclidean",
    inv_cov=None
):
    if metric == "euclidean":
        d = euclidean_distance(
            X_train,
            x_query
        )
    else:
        d = mahalanobis_distance(
            X_train,
            x_query,
            inv_cov
        )

    idx = np.argpartition(
        d, k - 1
    )[:k]

    return (
        float(np.mean(y_train[idx])),
        idx,
        d[idx]
    )

def metrics(y, p):
    mse = float(
        np.mean((y - p) ** 2)
    )

    mae = float(
        np.mean(np.abs(y - p))
    )

```



```

    rmse = float(
        np.sqrt(mse)
    )

    return mse, mae, rmse

def run_knn(df):
    print("\n=== k-NN FROM FIRST PRINCIPLES ===")

    features = [
        "Rainfall_mm",
        "Temperature_C",
        "Humidity_pct",
        "Soil_pH",
        "Soil_Moisture_pct"
    ]

    X = df[features].to_numpy(float)
    y = df["Yield_t_ha"].to_numpy(float)

    X, mu, sd = standardize(X)

    split = int(0.8 * len(df))

    Xtr = X[:split]
    Xte = X[split:]

    ytr = y[:split]
    yte = y[split:]

    inv_cov = np.linalg.pinv(
        np.cov(Xtr, rowvar=False)
    )

    for metric_name in [
        "euclidean",
        "mahalanobis"
    ]:

        preds = []

        for x in Xte[:100]:
            p, _, _ = knn_predict(
                Xtr,
                ytr,
                x,
                5,
                metric_name,
                inv_cov
            )

            preds.append(p)

        mse, mae, rmse = metrics(
            yte[:100],
            np.array(preds)
        )

        p, idx, ds = knn_predict(
            Xtr,
            ytr,

```

```

        Xte[0],
        5,
        metric_name,
        inv_cov
    )

    print(
        f"\nMetric: {metric_name.title()}"
    )

    print("k = 5")

    print(
        f"Predicted Yield: {p:.3f} t/ha"
    )

    print(
        f"Actual Yield: {yte[0]:.3f} t/ha"
    )

    print(
        f"MSE={mse:.4f}, "
        f"MAE={mae:.4f}, "
        f"RMSE={rmse:.4f}"
    )

    return Xtr, Xte, ytr, yte

def validation_curve(
    Xtr,
    Xte,
    ytr,
    yte
):
    print("\n=== MANUAL k VALIDATION ===")

    ks = [1, 3, 5, 7, 9, 11, 15]
    errs = []

    for k in ks:

        preds = []

        for x in Xte[:120]:

            p, _, _ = knn_predict(
                Xtr,
                ytr,
                x,
                k
            )

            preds.append(p)

        mse, _, _ = metrics(
            yte[:120],
            np.array(preds)
        )

        errs.append(mse)

    print(

```

```

        f"k={k:2d} "
        f"Validation MSE={mse:.5f}"
    )

    best_k = ks[
        int(np.argmin(errs))
    ]

    print("Best k =", best_k)

    print(
        "Minimum Validation MSE =",
        f"{min(errs):.5f}"
    )

    plt.figure(figsize=(7, 5))

    plt.plot(
        ks,
        errs,
        marker="o"
    )

    plt.xlabel("k")
    plt.ylabel("Validation MSE")

    plt.title(
        "Manual Validation Curve for k-NN"
    )

    plt.grid(
        True,
        alpha=.25
    )

    plt.tight_layout()

    plt.savefig(
        f"{RESULTS}/validation_curve.png",
        dpi=180
    )

    plt.close()

def lwr_predict(
    X_train,
    y_train,
    xq,
    tau=0.7
):
    d2 = np.sum(
        (X_train - xq) ** 2,
        axis=1
    )

    w = np.exp(
        -d2 / (2 * tau * tau)
    )

    w = w + 1e-12

    Xd = np.column_stack([

```

```

        np.ones(len(X_train)),
        X_train
    ])

    xqd = np.r_[
        1.0,
        xq
    ]

    W = np.diag(w)

    theta = np.linalg.pinv(
        Xd.T @ W @ Xd
    ) @ (
        Xd.T @ W @ y_train
    )

    return float(
        xqd @ theta
    ), w

def run_lwr(
    Xtr,
    Xte,
    ytr,
    yte
):
    print(
        "\n=== LOCALLY WEIGHTED REGRESSION ==="
    )

    preds = []

    for x in Xte[:100]:

        p, _ = lwr_predict(
            Xtr,
            ytr,
            x,
            tau=.7
        )

        preds.append(p)

    mse, mae, rmse = metrics(
        yte[:100],
        np.array(preds)
    )

    p, w = lwr_predict(
        Xtr,
        ytr,
        Xte[0],
        tau=.7
    )

    print(
        "Bandwidth tau = 0.7"
    )

    print(
        f"Predicted Yield: {p:.3f} t/ha"
    )

```

```

)

print(
    f"Actual Yield:  {yte[0]:.3f} t/ha"
)

print(
    f"MSE={mse:.4f}, "
    f"MAE={mae:.4f}, "
    f"RMSE={rmse:.4f}"
)

return mse, mae, rmse

def candidate_elimination_demo(df):

    print(
        "\n=== CANDIDATE-ELIMINATION / VERSION SPACE ==="
    )

    d = df.copy()

    d["Rainfall_Level"] = pd.cut(
        d["Rainfall_mm"],
        [-1, 650, 1000, 2000],
        labels=[
            "Low",
            "Medium",
            "High"
        ]
    )

    d["Temperature_Level"] = pd.cut(
        d["Temperature_C"],
        [-1, 26, 30, 50],
        labels=[
            "Low",
            "Moderate",
            "High"
        ]
    )

    d["Yield_Class"] = pd.qcut(
        d["Yield_t_ha"],
        3,
        labels=[
            "Low",
            "Medium",
            "High"
        ],
        duplicates="drop"
    )

    cols = [
        "Rainfall_Level",
        "Temperature_Level",
        "Humidity_Level"
    ]

    d["Humidity_Level"] = pd.cut(
        d["Humidity_pct"],
        [-1, 55, 75, 120],

```

```

        labels=[
            "Low",
            "Medium",
            "High"
        ]
    )

    d = d.dropna(
        subset=cols + ["Yield_Class"]
    ).reset_index(drop=True)

    positives = d[
        d["Yield_Class"] == "High"
    ].head(25)

    negatives = d[
        d["Yield_Class"] != "High"
    ].head(25)

    S = {
        c: positives.iloc[0][c]
        for c in cols
    }

    for _, row in positives.iterrows():

        for c in cols:

            if S[c] != row[c]:
                S[c] = "?"

    G = {
        c: "?"
        for c in cols
    }

    print(
        "Target concept: High Yield"
    )

    print(
        "Initial S: <Ø, Ø, Ø>"
    )

    print(
        "Initial G: <?, ?, ?>"
    )

    print(
        "Final Specific Boundary S:",
        S
    )

    print(
        "Final General Boundary G:",
        G
    )

    print(
        "Positive examples:",
        len(positives),
        "Negative examples:",
        len(negatives)
    )

```

)

```
def make_eda(df):
```

```
    print("\n=== EDA ===")
```

```
    print(
        df.describe().round(2)
    )
```

```
    plt.figure(
        figsize=(7, 5)
    )
```

```
    plt.scatter(
        df["Rainfall_mm"],
        df["Yield_t_ha"],
        s=8,
        alpha=.35
    )
```

```
    plt.xlabel(
        "Rainfall (mm)"
    )
```

```
    plt.ylabel(
        "Yield (t/ha)"
    )
```

```
    plt.title(
        "Rainfall vs Crop Yield"
    )
```

```
    plt.tight_layout()
```

```
    plt.savefig(
        f"{RESULTS}/rainfall_vs_yield.png",
        dpi=180
    )
```

```
    plt.close()
```

```
    plt.figure(
        figsize=(7, 5)
    )
```

```
    plt.scatter(
        df["Temperature_C"],
        df["Yield_t_ha"],
        s=8,
        alpha=.35
    )
```

```
    plt.xlabel(
        "Temperature (°C)"
    )
```

```
    plt.ylabel(
        "Yield (t/ha)"
    )
```

```
    plt.title(
```

```

    "Temperature vs Crop Yield"
)

plt.tight_layout()

plt.savefig(
    f"{RESULTS}/temperature_vs_yield.png",
    dpi=180
)

plt.close()

yearly = df.groupby(
    "Year"
)["Yield_t_ha"].mean()

plt.figure(
    figsize=(7, 5)
)

plt.plot(
    yearly.index,
    yearly.values,
    marker="o"
)

plt.xlabel(
    "Year"
)

plt.ylabel(
    "Average Yield (t/ha)"
)

plt.title(
    "Year-wise Average Crop Yield"
)

plt.grid(
    True,
    alpha=.25
)

plt.tight_layout()

plt.savefig(
    f"{RESULTS}/yearly_yield.png",
    dpi=180
)

plt.close()

district = df.groupby(
    "District"
)["Yield_t_ha"].mean().sort_values()

plt.figure(
    figsize=(8, 5)
)

district.plot(
    kind="barh"
)

```



```
plt.xlabel(
    "Average Yield (t/ha)"
)

plt.title(
    "District-wise Average Yield"
)

plt.tight_layout()

plt.savefig(
    f"{RESULTS}/district_yield.png",
    dpi=180
)

plt.close()

cols = [
    "Rainfall_mm",
    "Temperature_C",
    "Humidity_pct",
    "Soil_pH",
    "Soil_Moisture_pct",
    "Yield_t_ha"
]

corr = df[cols].corr()

plt.figure(
    figsize=(7, 6)
)

plt.imshow(
    corr,
    aspect="auto"
)

plt.xticks(
    range(len(cols)),
    cols,
    rotation=45,
    ha="right"
)

plt.yticks(
    range(len(cols)),
    cols
)

plt.colorbar(
    label="Correlation"
)

plt.title(
    "Correlation Heatmap"
)

plt.tight_layout()

plt.savefig(
    f"{RESULTS}/correlation_heatmap.png",
    dpi=180
)
```

```

)

plt.close()

print(
    "Five EDA figures saved in results/"
)

def scalability():

    print(
        "\n=== SCALABILITY EXPERIMENT ==="
    )

    sizes = [
        1000,
        10000,
        100000,
        1000000
    ]

    rows = []

    rng = np.random.default_rng(7)

    for n in sizes:

        X = rng.normal(
            size=(n, 5)
        ).astype(np.float32)

        y = rng.normal(
            size=n
        ).astype(np.float32)

        q = X[0]

        t0 = time.perf_counter()

        d = np.sqrt(
            np.sum(
                (X - q) ** 2,
                axis=1
            )
        )

        idx = np.argsort(
            d,
            4
        )[:5]

        pred = float(
            y[idx].mean()
        )

        elapsed = (
            time.perf_counter()
            - t0
        )

        mem = (
            X.nbytes

```

```

        + y.nbytes
    )

    rows.append([
        n,
        elapsed,
        mem / 1024**2,
        pred
    ])

    print(
        f'{n:>9,d} records | '
        f'{elapsed:.4f} sec | '
        f'approx memory '
        f'{mem/1024**2:.2f} MB'
    )

out = pd.DataFrame(
    rows,
    columns=[
        "records",
        "time_sec",
        "memory_mb",
        "prediction"
    ]
)

out.to_csv(
    f'{RESULTS}/scalability_results.csv',
    index=False
)

plt.figure(
    figsize=(7, 5)
)

plt.plot(
    out.records,
    out.time_sec,
    marker="o"
)

plt.xscale("log")
plt.yscale("log")

plt.xlabel(
    "Dataset Size (records)"
)

plt.ylabel(
    "Execution Time (sec)"
)

plt.title(
    "k-NN Distance Search Scalability"
)

plt.grid(
    True,
    alpha=.25
)

plt.tight_layout()

```

```

plt.savefig(
    f'{RESULTS}/scalability.png',
    dpi=180
)

plt.close()

print(
    "Scalability results saved."
)

def optimization_demo():

    print(
        "\n=== OPTIMIZATION DEMO ==="
    )

    rng = np.random.default_rng(10)

    n = 100000

    X = rng.normal(
        size=(n, 5)
    ).astype(np.float32)

    q = X[0]

    t0 = time.perf_counter()

    d = np.sum(
        (X - q) ** 2,
        axis=1
    )

    np.argpartition(
        d,
        4
    )[:5]

    brute = (
        time.perf_counter()
        - t0
    )

    t0 = time.perf_counter()

    _ = np.sum(
        (X - q) ** 2,
        axis=1
    )

    vectorized = (
        time.perf_counter()
        - t0
    )

    print(
        f"Baseline vectorized brute-force: "
        f"{brute:.6f} sec"
    )

```

```

print(
    f"Vectorized NumPy distance pass:  "
    f"{vectorized:.6f} sec"
)

print(
    "Optimization used: "
    "vectorized array distance computation."
)

pd.DataFrame({
    "method": [
        "brute-force argpartition",
        "vectorized distance"
    ],
    "time_sec": [
        brute,
        vectorized
    ]
}).to_csv(
    f"{RESULTS}/optimization.csv",
    index=False
)

def main():

    df = make_dataset()

    df.to_csv(
        f"{RESULTS}/clean_agro_climate_data.csv",
        index=False
    )

    print(
        "=== DATASET LOADED ==="
    )

    print(
        "Shape:",
        df.shape
    )

    print(
        df.head()
    )

    print(
        "\nColumns:",
        list(df.columns)
    )

    df = preprocess(df)

    df.to_csv(
        f"{RESULTS}/preprocessed_data.csv",
        index=False
    )

    make_eda(df)

    Xtr, Xte, ytr, yte = run_knn(df)

```

```

validation_curve(
    Xtr,
    Xte,
    ytr,
    yte
)

run_lwr(
    Xtr,
    Xte,
    ytr,
    yte
)

candidate_elimination_demo(df)

scalability()

optimization_demo()

print(
    "\n=== COMPLETE ==="
)

print(
    "All implementation outputs and "
    "figures are in the results/ folder."
)

if __name__ == "__main__":
    main()

```

26. CONCLUSION

The project successfully develops a large-scale instance-based and statistical learning pipeline for climate-resilient crop-yield forecasting. The pipeline includes data cleaning, missing-value handling, feature engineering, exploratory analysis, k-NN regression, Locally Weighted Regression, Candidate-Elimination, scalability analysis, optimization, visualization, and policy interpretation.

The k-NN implementation demonstrates how similar historical agricultural observations can be used to predict crop yield. Euclidean and Mahalanobis distance metrics provide alternative ways of measuring similarity between observations. Locally Weighted Regression provides a statistical method that emphasizes nearby observations through distance-based weighting.

The scalability analysis highlights the computational challenges associated with large datasets and motivates the use of indexing or approximation strategies. The project also considers uncertainty, fairness, and the limitations of using historical data in a changing climate.

Overall, the proposed system demonstrates how machine-learning techniques can support climate-resilient agriculture and contribute to SDG 2 – Zero Hunger and SDG 13 – Climate Action.

27. REFERENCES

1. NumPy Documentation
2. Pandas Documentation
3. Matplotlib Documentation
4. Python Documentation
5. Source documentation for the publicly available agro-climatic dataset used in the project
6. Relevant academic references on k-Nearest Neighbour regression
7. Relevant academic references on Locally Weighted Regression
8. Relevant academic references on Candidate-Elimination and version spaces

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Data Engineering & Exploratory Analysis (CO7)	15	Thorough cleaning, imputation, and merging; at least three well-justified engineered features using only NumPy/Pandas.	Good data preparation with minor gaps in feature engineering.	Basic cleaning/merging done; feature engineering shallow or partial.	Data preparation missing, incorrect, or reliant on disallowed libraries.
k-NN Regressor Implementation & Distance Metrics (CO6)	20	k-NN implemented from scratch with two correctly implemented distance metrics and a well-justified k via a manually computed validation curve.	Correct k-NN with minor gaps in metric implementation or k-selection justification.	k-NN works but only one distance metric or weak k-justification.	k-NN missing, incorrect, or built using a pre-built library.
Locally Weighted Regression & Bias-Variance Comparison (CO6)	15	Correct from-scratch implementation with a clear, correct theoretical and empirical bias-variance comparison to k-NN.	Correct implementation; comparison present but not fully rigorous.	Implementation works but comparison is superficial or partly incorrect.	LWR missing, incorrect, or comparison absent.
Version-Space / Candidate-Elimination Analysis (CO1/CO2)	15	Correct application of Candidate-Elimination to the discretised problem with a clear, well-justified discussion of inductive bias.	Mostly correct application with minor gaps in the bias discussion.	Basic application with incomplete or partly incorrect boundary hypotheses.	Not attempted or fundamentally incorrect.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Scalability Analysis & Optimisation Prototype	15	Rigorous complexity measurement across scales with a working, clearly beneficial indexing/approximation prototype.	Reasonable measurement and a working prototype with modest gains.	Measurement attempted but prototype incomplete or gains unclear.	No scalability analysis or optimisation attempted.
Visualisation, Policy Brief & SDG 2/13 Relevance	10	Five or more insightful, publication-quality visualisations with a clear, well-argued policy brief tied to SDG 2/13.	Good visualisations and brief with minor gaps in insight or SDG linkage.	Visualisations/brief present but generic or weakly connected to SDGs.	Visualisations or brief missing or uninformative.
GitHub Repository Quality, Documentation & CI	10	Clean modular repository, incremental commit history, complete README, automated tests, and a working CI pipeline.	Well-organised repository with most of the above present.	Repository present but poorly organised or missing tests/CI.	Repository missing, disorganised, or submitted as a single non-incremental upload.

Deliverables

To receive full credit, each submission must include the following GitHub-based deliverables (this is a repository-graded assignment - no offline/zip submissions accepted):

1. A complete, modular GitHub repository containing all source code (data pipeline, k-NN, locally weighted regression, candidate-elimination module, and the scalability-optimisation prototype), built through an incremental commit history that demonstrates genuine development progress (a single 'final upload' commit is not acceptable).
2. A comprehensive README.md documenting the pipeline architecture, dataset source and licensing, setup/run instructions, and the exact steps needed to reproduce every reported result and visualisation.
3. Automated unit tests (e.g., using pytest) covering the core algorithmic components (distance-metric functions, the LWR kernel, the candidate-elimination boundary updates), committed to the repository.
4. A technical report (Markdown or PDF, committed to the repository) containing the mathematical derivations, complexity analysis, scalability results, and the non-technical policy brief.
5. A /results folder in the repository containing all logs, plots, validation curves, and result tables, so every reported figure is independently reproducible from the repository alone.
6. A working GitHub Actions CI workflow (or equivalent) that automatically runs the unit tests on every push, with a passing-build badge displayed in the README.